Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 17

Sorting

Sorting

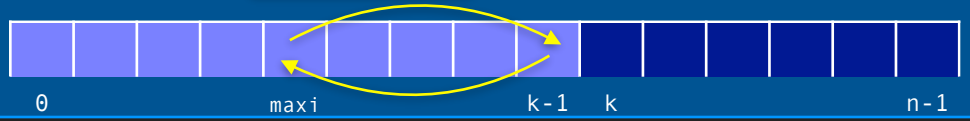
- There are several sorting algorithms with asymptotic worst-case costs ranging from $O(n^2)$ to $O(n \log n)$
  - In-place or not
    - In-place: $O(1)$ additional space. Input rearranged into sorted order
  - Stable or not
    - Stable: If there are equal valued keys, they appear in the output in the same order in which they are in the input
  - In practice, concerns beyond worst-case asymptotic behaviour
    - Average case costs. Performance on small inputs.

Selection Sort

- Recall the heap sort approach:
  - At each iteration identify the max, and move it to its final location
- Selection sort is the simplest implementation of this

```
void selection_sort(std::vector<int>& A) {
  bool sorted = false;   // optional flag to check for early exit
  for(int k = A.size(); k>1 && !sorted ; --k) { // k items remain to be sorted
    sorted = true;
    for (int i=1, maxi=0; i<k; ++i) {//scan remaining items for max's index
      if(A[i] >= A[maxi]) maxi=i; else sorted = false;
    }
    if(maxi!=k-1) std::swap(A[k-1],A[maxi]);
  }
}
```

Makes it unstable



- $O(n^2)$ comparisons. Only O(n) assignments (swaps).
- Not a stable sort (equal items can change order when max item swapped out)
- Can exit early if the array becomes sorted sooner

Selection Sort → Heap Sort

- Instead of O(n) time scan to find max, use a *max-heap*
- O(log n) time to remove the max item; O(n log n) time overall

```
void heap_sort(std::vector<int>& A) {
  makeMaxHeap(A);  // a function that turns A into a max-heap in place
  for(int k = A.size(); k>1 ; --k) { // k items remain to be sorted
    int x = A[k-1]; A[k-1] = A[0];
    bubbleDown(A,k-1,0,x); // A[0] is bubbled down in heap A[0..k-1] with key x
  }
}
```

Sending A[k-1] to the root (and bubbling down) makes it unstable

specify how many items in A are part of the heap

location to bubbleDown and its new key value

- Exercise: In-place implementations of the functions makeMaxHeap and bubbleDown as required above
- In-place, and not stable (like selection sort)

Insertion Sort

- A slightly different idea yielding a stable sort
  - We will still maintain a suffix in sorted order
  - But the sorted items needn't be in their final locations
  - In each iteration absorb the last unsorted item (instead of max)
    - *Insert it into* the sorted part (sliding items less than it)

```
void insertion_sort(std::vector<int>& A) {
  for(int k = A.size(); k>0; --k) { // k items remain to be absorbed
    int i, x = A[k-1];              // will absorb x into the sorted part
    for (i=k; i<A.size() && A[i] < x; ++i) // scan for x's spot
      A[i-1]=A[i];                  // while sliding items less than x to the left
    A[i-1]=x; //outside the loop, i-1 is where x settles in.
  }
}
```

Important for stability: use <, not <=



- Stable, in-place sorting

Tree Sort

- Same as insertion sort, but instead of keeping the sorted part in (the suffix of) a vector, keep it in a balanced BST

```
void tree_sort(std::vector<int>& A) {
  balancedBST tree; // not in-place any more
  for(int k = 0; k < A.size(); ++k) { // k items remain to be absorbed
    tree.insert(A[k]);
  }
  tree.inorder(A); // overwrite the vector with tree's in-order sequence
}
```

For stability, requires that a duplicate key inserted later goes to the right of the existing key (and stays so while balancing)

- O(n log n) time, stable
- Needs O(n) additional space: not in-place

Attendance Break!

Login using CSE LDAP id at

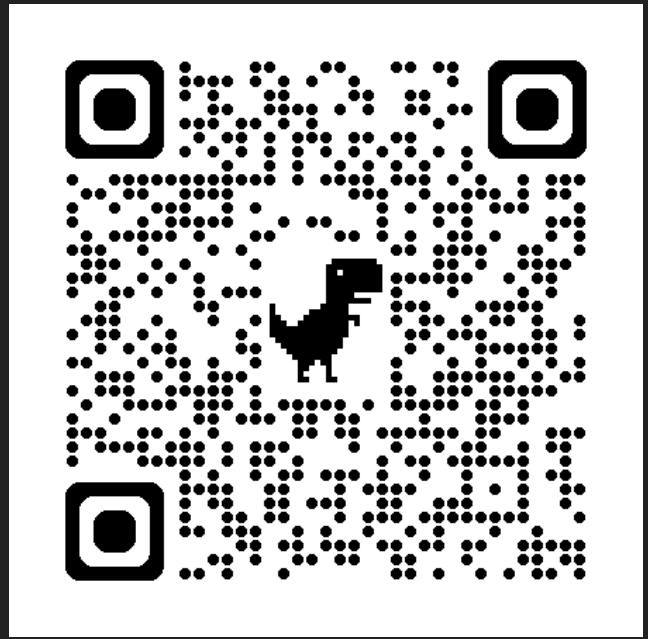
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Divide-and-Conquer

- Some *algorithms* use a Divide-and-Conquer strategy
  (a.k.a. Divide-Conquer-and-Combine)
  - A problem instance is divided into two or more smaller problems
  - The smaller instances are solved recursively
  - The results are then combined to get the result for the original instance
- Two examples: Merge Sort and Quicksort

```
resultType f(inputType x) {

  if (baseCase(x))
    return handleBaseCase(x);

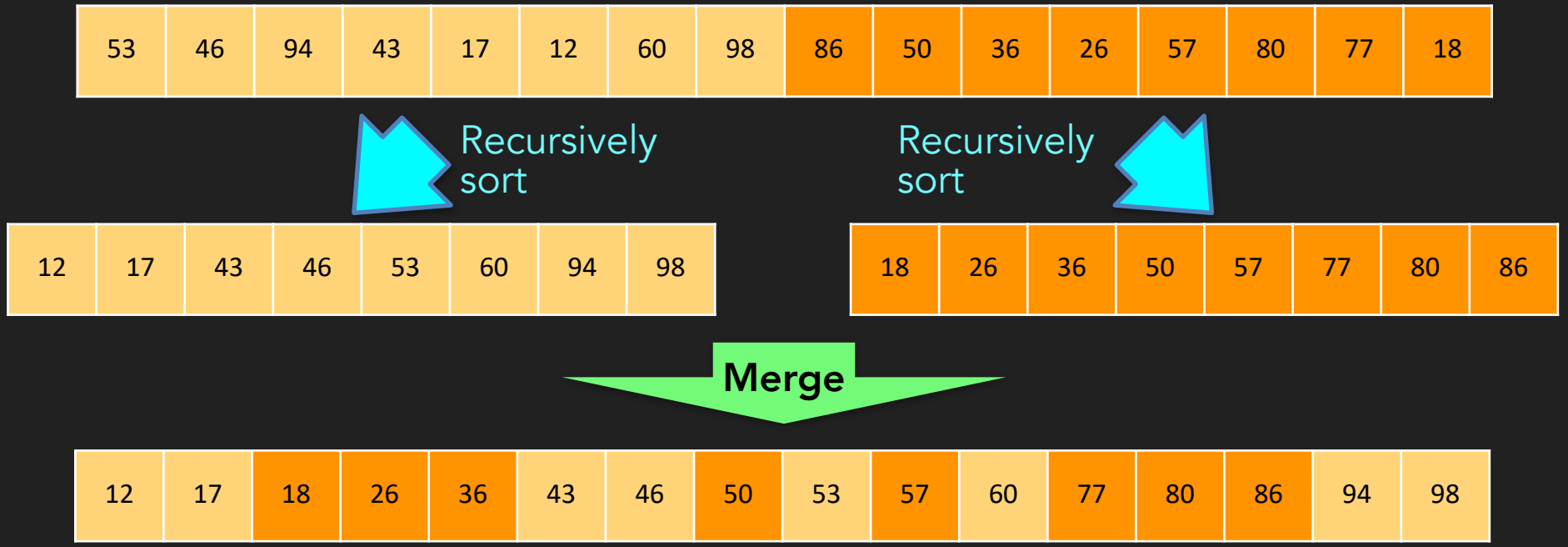
  inputType x1, x2;
  Divide(x,x1,x2);

  resultType y1, y2;
  y1 = f(x1); y2 = f(x2);

  return Combine(y1,y2);
}
```

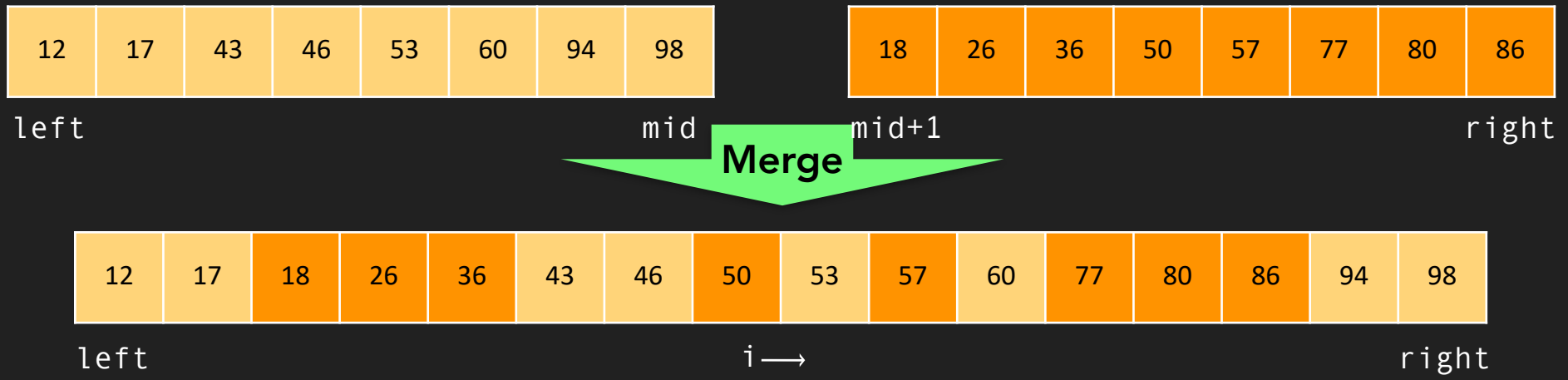
Merge Sort

- Split into two (almost) equal halves, and recursively sort each half
- Merge the two sorted arrays into a single sorted array



Merge Sort

```
// merge X[left..mid] and X[mid+1..right] into Y[left..right]
void merge(const vector<int>& X, int left, int mid, int right, vector<int>& Y) {
     int L = left, R = mid+1;   // L,R: next indices of left/right halves
     for(int i=left; i <= right; ++i) {
          if(L <= mid && (R > right || X[L] <= X[R])) Y[i] = X[L++];    // copy from left
          else Y[i] = X[R++];                                           // copy from right
     }
}
```



Merge Sort

```
void sort (const std::vector<int>& in, int left,
           int right, std::vector<int>& out,
                      std::vector<int>& aux) {

  if (left == right)
    out[left] = in[left];
  if (left >= right)
    return;

  int mid = (left+right)/2;

  sort(in,left,mid,aux,out);  // outputs into aux
  sort(in,mid+1,right,aux,out);

  merge(aux,left,mid,right,out); // aux is input
}
```

Output will be in `out[left,...,right]`. A temporary vector `aux` is passed (to avoid allocating Θ(n) space in each recursive call)

Total extra space is O(n) + O(log n) = O(n)

```
resultType f(inputType x) {

  if (baseCase(x))
    return handleBaseCase(x);

  inputType x1, x2;
  Divide(x,x1,x2);

  resultType y1, y2;
  y1 = f(x1); y2 = f(x2);

  return Combine(y1,y2);
}
```

Merge Sort

```
void sort (const std::vector<int>& in, int left,
           int right, std::vector<int>& out,
                      std::vector<int>& aux) {


  if (left == right)
    out[left] = in[left];
  if (left >= right)
    return;

  int mid = (left+right)/2;

  sort(in,left,mid,aux,out);
  sort(in,mid+1,right,aux,out);

  merge(aux,left,mid,right,out);
}
```

- Time?
  - T(n) = 2T(n/2) + O(n)
  - O(n log n)

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 18

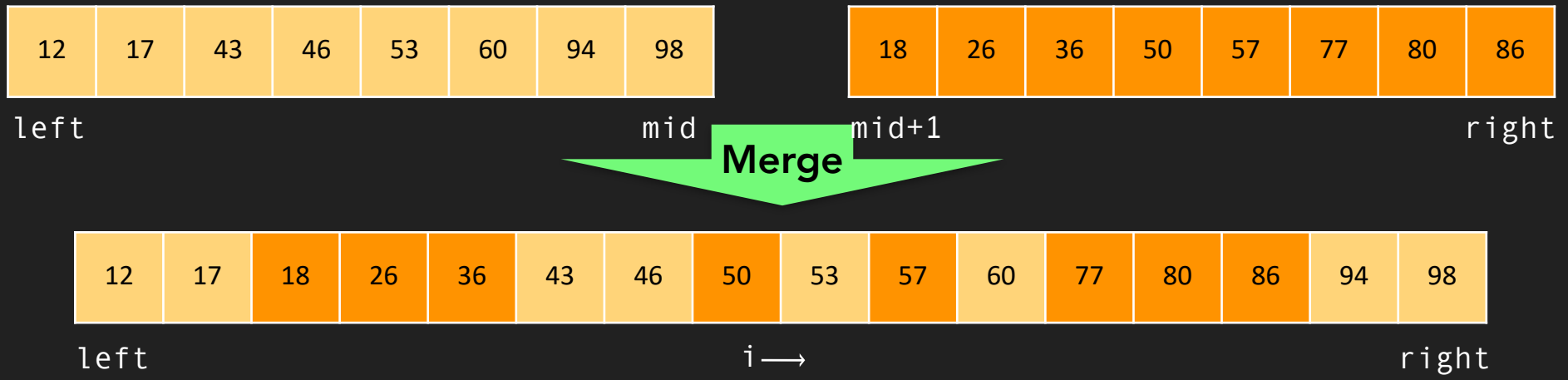
Sorting (ctd.)

Sorting

- So far:
  - Selection sort and Heapsort:
    - $O(n^2)$ vs $O(n \log n)$. Both in-place, not stable.
  - Insertion sort and Tree sort:
    - $O(n^2)$, in-place vs $O(n \log n)$, not in-place. Both stable.
  - Merge sort
    - $O(n \log n)$. Not in-place. Stable.
    - Let's recap

Merge Sort

```
// merge X[left..mid] and X[mid+1..right] into Y[left..right]
void merge(const vector<int>& X, int left, int mid, int right, vector<int>& Y) {
     int L = left, R = mid+1;   // L,R: next indices of left/right halves
     for(int i=left; i <= right; ++i) {
          if(L <= mid && (R > right || X[L] <= X[R])) Y[i] = X[L++];    // copy from left
          else Y[i] = X[R++];                                           // copy from right
     }
}
```



Merge Sort

```
void sort (const std::vector<int>& in, int left,
           int right, std::vector<int>& out,
                      std::vector<int>& aux) {

  if (left == right)
    out[left] = in[left];
  if (left >= right)
    return;

  int mid = (left+right)/2;

  sort(in,left,mid,aux,out);  // outputs into aux
  sort(in,mid+1,right,aux,out);

  merge(aux,left,mid,right,out); // aux is input
}
```

Output will be in `out[left,...,right]`. A temporary vector `aux` is passed (to avoid allocating Θ(n) space in each recursive call)

Total extra space is O(n) + O(log n) = O(n)

- Time?
  - T(n) = 2T(n/2) + O(n)
  - O(n log n)

Merge Sort

```
void sort (const std::vector<int>& in, int left,
           int right, std::vector<int>& out,
                      std::vector<int>& aux) {

  if (left == right)
    out[left] = in[left];
  if (left >= right)
    return;

  int mid = (left+right)/2;

  sort(in,left,mid,aux,out);  // outputs into aux
  sort(in,mid+1,right,aux,out);

  merge(aux,left,mid,right,out); // aux is input
}
```

- Non-recursive version?
  - Need to sort both child segments before sorting parent segment
  - Any child-before-parent order suffices
    - Recall: makeHeap
- Bottom-up ("reverse breadth-first" order) will work!

Merge Sort: Non-Recursive

```
void sort (std::vector<int>& in) {
  int N = in.size(); std::vector<int> aux(N);
  auto A = &in; auto B = &aux; // for conveniently switching between the two
  // imagine a nearly complete binary tree, where i-th node at height t
  // covers a block of T=2^t indices from i*T to (i+1)*T-1
  for(int t=0, T=1; T < N; T *= 2, ++t) { // merge pairs of T-long blocks
    for(int left=0; left < N; left += 2*T) {
      int mid = std::min(N-1,left+T-1), right = std::min(N-1,left+2*T-1);
      merge(*A,*B,left,mid,right); // if empty right half, just copies A to B
    }
   std::swap(A,B); // merge's output (B) should become its input (A) next time
  }
  // sorted array is in A
  if(A != &in) in = aux; // if necessary, copy aux to in
}
```

O(1) option available:
`in = std::move(aux);`

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Quicksort

- Another divide-and-conquer sorting algorithm
  - Almost in-place. Not stable.
- Divide into two lists, S_1 = [items < pivot] and S_2 = [items > pivot]
- Combine: [quicksort(S_1) [items == pivot] quicksort(S_2)]
- How is pivot picked? Various approaches
  - Pick the first element: Worst case $O(n^2)$
  - Pick a random element: Worst case O(n log n) with high probability
  - Pick the "median of medians": Worst case O(n log n)
  - Heuristic choices: Worst case $O(n^2)$, but often better than that
- Hybrid models: switch to another sorting if taking too long

E.g., when the data is sorted or reverse-sorted

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 19

Sorting (ctd.)

Sorting

- So far:
  - Selection sort and Heapsort:
    - $O(n^2)$ vs $O(n \log n)$. Both in-place, not stable.
  - Insertion sort and Tree sort:
    - $O(n^2)$, in-place vs $O(n \log n)$, not in-place. Both stable.
  - Merge sort
    - $O(n \log n)$. Not in-place. Stable.
- Today: Quicksort

Quicksort

- Another divide-and-conquer sorting algorithm
  - Almost in-place. Not stable.
- Divide into two lists, S_1 = [items < pivot] and S_2 = [items > pivot]
- Combine: [quicksort(S_1) [items == pivot] quicksort(S_2)]
- How is pivot picked? Various approaches
  - Pick the first element: Worst case $O(n^2)$
  - Pick a random element: Worst case O(n log n) with high probability
  - Pick the "median of medians": Worst case O(n log n)
  - Heuristic choices: Worst case $O(n^2)$, but often better than that
- Hybrid models: switch to another sorting if taking too long

E.g., when the data is sorted or reverse-sorted

With high probability, close to the ideal situation where T(n) = 2T(n/2) + O(n)

Deterministically, close enough to the ideal

Quicksort: Using Aux Space

```
void qsort (std::vector<int>& A) {
  int N = A.size();
  if(N<=1) return;
  int pivot = A[0]; // may use alternate logic
  std::vector<int> L, E, G; // Less, Equal and Greater
  for(int i=0; i<N; ++i) { // scan and populate the 3 arrays
    if (A[i] < pivot) L.push_back(A[i]);
    else if (A[i] > pivot) G.push_back(A[i]);
    else E.push_back(A[i]);
  }
  qsort(L); qsort(G);
  // copy L, E, G to A
  std::copy(L.begin(), L.end(), A.begin());
  std::copy(E.begin(), E.end(), A.begin()+L.size());
  std::copy(G.begin(), G.end(), A.end()-G.size());
}
```

Quicksort: In-Place Pivoting

```
void qsort (std::vector<int>& A, int left, int right) {
  if(left>=right) return;
  int pivot = A[left]; // may use alternate (in-place) logic
  int L=left, G=right;
  for(int i=left; i<=G; ) {
    if (A[i] < pivot) std::swap(A[L++],A[i++]);
    else if (A[i] > pivot) std::swap(A[G--],A[i]);
    else ++i;
  }
  qsort(A,left,L-1); qsort(A,G+1,right);
}
```



- Almost in-place: The data itself stays in the original container
  - Recursion uses a stack to store left and right (can't predict them)
  - Recursion depth can be made O(log n) by the choice of pivot

Merge Sort and Quicksort

- Both are practically important algorithms: Quicksort used in `std::sort` and Merge Sort in `std::stable_sort`
- In practice, combined with several optimisations
  - The base-case is not one element, but a small enough array . Then it switches to, say, insertion sort (in-place and stable).
  - `std::sort`, uses a more complex (deterministic) choice of pivot, and if the recursion depth goes well beyond log n, switches to heapsort
  - `std::stable_sort`, if not enough memory for standard merge sort, falls back to a more complex, but in-place, $O(n \log^2 n)$-time merge algorithm
  - Heuristics to exploit pre-sorted subsequences and patterns

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



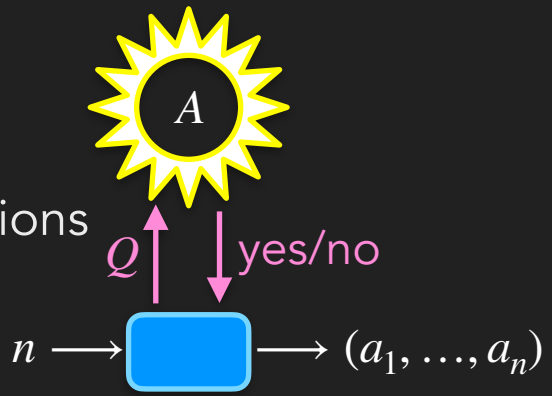
Lower Bound: $\Omega(n \log n)$ Comparisons

- General purpose sorting algorithms are "comparison-based"
  - e.g., `std::sort` and `std::stable_sort` work, by default, on any data-type with `operator<` (and supports moving)
- **Claim:** Any comparison-based sorting algorithm will need to make, in the worst-case, at least $cn \log n$ comparisons, where c is a constant

  $(\exists n_0, c > 0 \; \forall n > n_0 \quad N_{\text{comp}}(n) \geq cn \log n)$
  - $\Omega(n \log n)$ comparisons
- There are some non-comparison-based sorting algorithms, exploiting additional guarantees/structure on the data
  - E.g., bucket sort and radix sort (can be faster for sorting numbers in certain ranges)

Lower Bound: $\Omega(n \log n)$ Comparisons

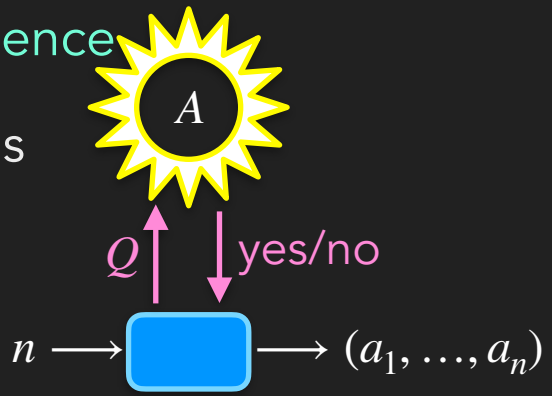
- Formalising a comparison-based sorting algorithm:
  - Given n and an "oracle" that answers boolean questions about an n-item vector A (e.g. $A[i] < A[j]$?)
  - To output indices $(a_1, \ldots, a_n)$ s.t. $A[a_1] \leq \ldots \leq A[a_n]$



- Fix n and a correct sorting algorithm. How many queries should it ask the oracle?
- Key observation: The final output is completely determined by the response sequence
  - Can reproduce the output by running the algorithm and feeding it responses from this sequence (without needing to know A)

Lower Bound: $\Omega(n \log n)$ Comparisons

- Output completely determined by the response sequence
- Suppose the algorithm makes at most k oracle queries
  - W.l.o.g, assume it always makes <u>exactly</u> k queries
    - Because the algorithm can be modified so that it makes additional dummy queries at the end if necessary
  - Number of possible response sequences from the oracle is at most 2^k
  - So number of distinct outputs is $\leq 2^k$
- But number of distinct outputs by a fully correct algorithm should be $n!$
- $n! \leq 2^k \implies k \geq \log_2(n!) \geq \log_2((n/2)^{n/2})$



Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 20

Sorting Networks

Sorting

- So far:
  - Selection sort and Heap Sort: $O(n^2)$ vs $O(n \log n)$. Both in-place, not stable. Heap Sort is a fall back for `std::sort`
  - Insertion sort and Tree Sort: $O(n^2)$, in-place vs $O(n \log n)$, not in-place. Both stable. Insertion sort used by `std::sort` and `std::stable_sort` to handle the base-case.
  - Merge Sort: $O(n \log n)$. Not in-place. Stable. Main component of `std::stable_sort`
  - Quicksort: $O(n \log n)$ on average, or using certain pivot selection schemes (not). Main component of `std::sort`

Bubble Sort

- Another simple $O(n^2)$-time in-place sorting algorithm (cf. selection sort and insertion sort)
  - Moves largest items to their final location, one by one (like selection sort)
  - Relative order of two items flipped only if required by sorting order, making it stable (unlike selection sort)

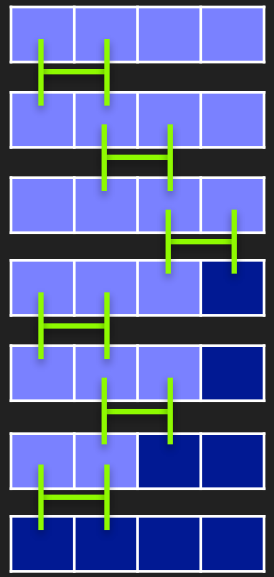
```
void sortpair(int& a, int& b) {
  if (a>b) std::swap(a,b)
}
void bubblesort (std::vector<int>& A) {
  for(int k=A.size(); k > 1; --k)
    for(int i=0; i<k-1; ++i)
      sortpair(A[i],A[i+1]);
}
```



Exercise:
1. Exit early, when already sorted.
2. Replace calls to sortpair with shifting to save on number of assignments.

Bubble Sort

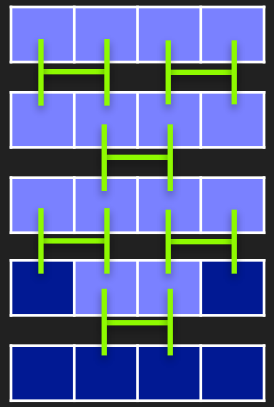
- The order of comparisons/moves doesn't depend on the data!
- Can build a "circuit" using "sortpair gates" (comparators) for sorting
  - Such a circuit is called a sorting network
- Complexity measures: Number of comparators (hardware cost), number of levels (time taken)



```
void sortpair(int& a, int& b) {
  if (a>b) std::swap(a,b)
}
void bubblesort (std::vector<int>& A) {
  for(int k=A.size(); k > 0; --k)
    for(int i=0; i<k-1; ++i)
      sortpair(A[i],A[i+1]);
}
```

Sorting Networks

- Can improve on the sorting network from bubble sort
  - Instead of $O(n^2)$ time, $O(n)$ levels enough
    - With $O(n)$ comparators at each level
    - Odd-Even Transposition Network
- How do we know this works?
  - Two-step argument:
    - **0-1 Principle:** Enough to analyse inputs in $\{0,1\}^n$
    - No error after n steps in a $\{0,1\}^n$ input



Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



0-1 Principle

- If a sorting network correctly sorts all 0-1 sequences, then it correctly sorts all inputs
- Contrapositive: If a sorting network is wrong on some input, then it must be wrong on some 0-1 sequence
- Proof: Suppose on input $(x_1,\ldots,x_n)$ it is wrong. Then some inputs a and $b$such that $a < b$ that ended up in the wrong order (b before a)
  - Consider input z such that $z_i = T(x_i)$ where $T(x) = \begin{cases} 0 \text{ if } x \leq a \\ 1 \text{ otherwise.} \end{cases}$
  - Inductively, at any level, results while sorting x and z, are related by $z^{(t)} = T(x^{(t)})$: because $\min(T(u),T(v)) = T(\min(u,v))$ and $\max(T(u),T(v))=T(\max(u,v))$.
  - At the final level, T(b)=1 before T(a)=0

n Steps for Odd-Even Transposition Network

- Two-step argument to prove a sorting network is correct:
  - **0-1 Principle:** For any sorting network, enough to analyse inputs in $\{0,1\}^n$
  - The given network correctly sorts every input in $\{0,1\}^n$
- Claim: The odd-even transposition network on n inputs, with n levels, correctly sorts every input in $\{0,1\}^n$
  - The rightmost 1 may or may not move in the first step (depending on if it is compared to the left or to the right); after that it will start *flowing*: i.e., move to the right at every step until it reaches its final position, n-1
  - Inductively: The kth-from-right 1 (whose final position is n-k) will start flowing after at most k steps, because it will not be blocked by a 1 to its right that is already flowing (but could stall once before starting to chase it)
  - All 1s reach final positions in at most n steps: ≤ k steps before flowing, and ≤ n-k steps in the flow

Sorting Networks

- Odd-even transposition network takes O(n) time using O(n) processors in parallel
  - As a circuit, needs $O(n^2)$ comparators
  - Can do much better! (cf. $\Omega(n \log n)$ lower bound)
- Batcher's Bitonic sorting & Batcher's Odd-Even Mergesort
  - $O(\log^2 n)$ time, using O(n) processors in parallel (total $O(n \log^2 n)$ comparators).
  - Practical (small constants hidden by O). Used in GPUs and circuits, for moderately large n.
- In principle, O(log n) time using O(n) processors enough!
  - But the hidden constants possible for the best known algorithms are in 1000s (where as $\log_2 n$ is *well under* 100 for all practical n

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 21

Non-Comparison-Based Sorting

Sorting

- So far:
  - Several comparison-based sorting algorithms
    - Merge sort, quicksort, heap sort, tree sort : $O(n \log n)$ time
    - Insertion sort, selection sort, bubble sort : $O(n^2)$ time
    - Sorting networks
      - Odd-even transposition: $O(n)$ depth with $O(n^2)$ comparators
      - $O(\log n)$ depth possible, $O(\log^2 n)$ depth practical
  - Lower Bound:
    - Any comparison-based sorting requires $\Omega(n \log n)$ comparisons
- Today: Non-comparison-based sorting: Counting sort, bucket sort, radix sort

(Basic) Counting Sort

- When the domain of keys to sort is small -- say, [0,ulim)

```
void basic_counting_sort (std::vector<unsigned int>& A, unsigned int ulim) {
  std::vector<int> counts(ulim,0); // one index per key, initialised to 0
  for(auto x : A)                  // compute count of each key in O(n) time
    ++counts[x];
  for(unsigned int x=0, index=0; x < ulim; x++) // for each key x in domain
    for(; counts[x] > 0; counts[x]--) // write counts[x] copies of x into A
      A[index++] = x;                 // keeping track of next index to use
}
```

- Extends to values in a small range `[llim,ulim)` (Exercise)
- Let `k = ulim - llim` be the size of the domain of the keys
- O(n+k) time. For small k, beats the Ω(n log n) lower bound for comparison sorts!
- O(k) auxiliary space

Collect Instead of Count

- When the items are structured, but with integer keys from a small range
  - Instead of just keeping the count of each key, we need to collect all the items with that key

```
struct item { unsigned int key; /* more data */ };
void collection_sort (std::vector<item>& A, unsigned int ulim) {
  std::vector<std::queue<item>> collection(ulim);  //per key a queue
  for(auto& x : A)                 //for each item in A (without making a copy)
    collection[x.key].push(std::move(x));          //move it into collection
  for(unsigned int x=0, index=0; x < ulim; x++)    //for each key x
    for(; !collection[x].empty(); collection[x].pop()) //for each item for x
      A[index++] = std::move(collection[x].front());   //move it into A
}
```

  - Stable (by collecting in a queue for each key)

Counting Sort : General Version

- Use counts to directly find the index in the output (no queues)

```
struct item { unsigned int key; /* more data */ };
void counting_sort (std::vector<item>& A, unsigned int ulim) {
  std::vector<unsigned int> offset(ulim,0);    // used for counting
  std::vector<item> output(A.size());
  for(auto& x : A) ++offset[x.key]; // after this, offset has counts
  for(unsigned int i=0, sum = 0; i < ulim; i++) {
    unsigned int tmp = offset[i];
    offset[i] = sum; sum += tmp;    // offset[i] has sum of previous counts
  }
  for(auto& x : A) // output[ offset[i] ] is where next item with key i goes
    output [ offset[x.key]++ ] = std::move(x); // move items from A to output
  A = std::move(output);            // finally move output vector to A
}
```

- O(n+k) time and auxiliary space. Stable.

Bucket Sort

- Can we generalise counting sort to large key domains (e.g., all of `int`, or `float`)?
  - By *bucketing*! `bucketof(key)` sends key to a small domain of "buckets"
  - Sort items into their respective buckets
    - Using counting sort: with `bucketof(key)` as the key
  - Within each bucket, sort (using any sorting algorithm)



Bucket Sort

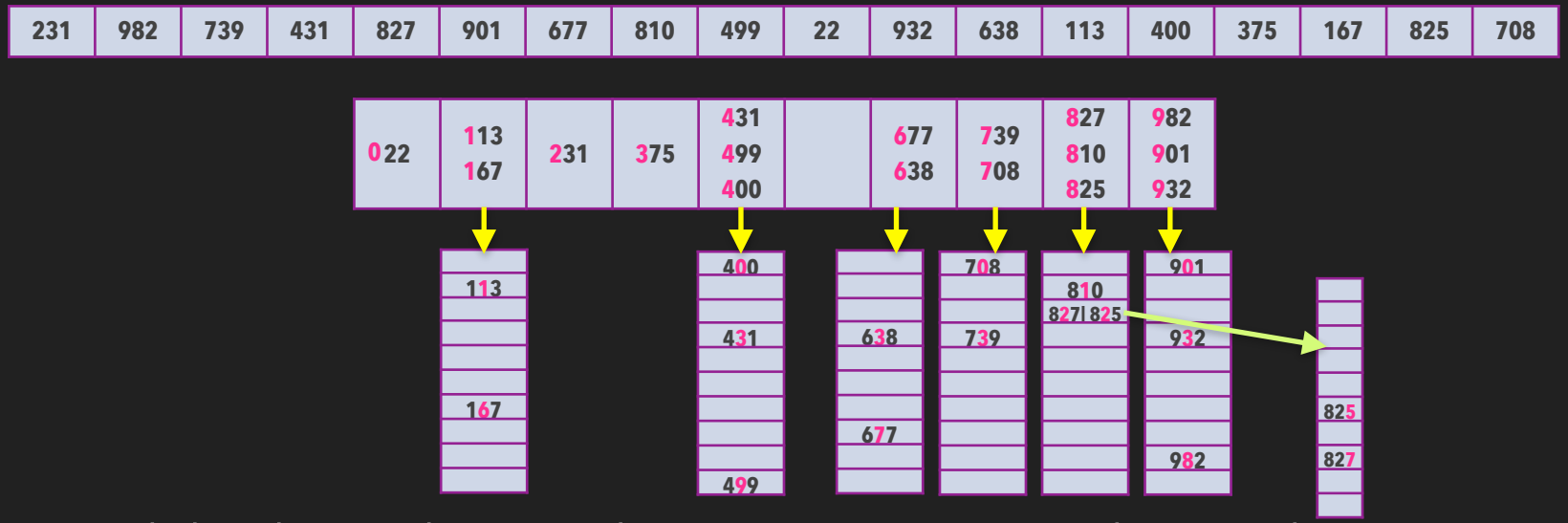
- If we use k buckets, takes O(n+k) auxiliary space
- Total time for sorting the buckets (using a comparison sort)?
  - If n = O(k) and each bucket has O(1) items, total time is O(n)
    - E.g., when the keys are uniformly spread throughout its domain
  - But can be as bad as Ω(n log n) if one bucket has Θ(n) items
- Parallelisable
  - Can collect items into buckets in parallel
  - Can sort each bucket in parallel
  - Can copy back from buckets into a vector in parallel (after offsets computed)

Bucket Sort

- What if Θ(n) items fall into one bucket?
  - If using a comparison based sorting algorithm for sorting within buckets, takes Θ(n log n)
- Can we still beat comparison based sorting?
  - Can use application-specific heuristics for defining buckets to avoid crowded buckets
  - Or, recursively, use bucket sort?
    - When a bucket has "too many" items, split it into smaller buckets
    - If the keys are "granular" (e.g., integers), not too many levels of recursion
    - Leads to another sorting algorithm! MSD Radix Sort

Most-Significant-Digit Radix Sort

- Consider d-digit numbers
  - First collect by MSD



- Each bucket with more than one item is sorted using the remaining digits

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 22

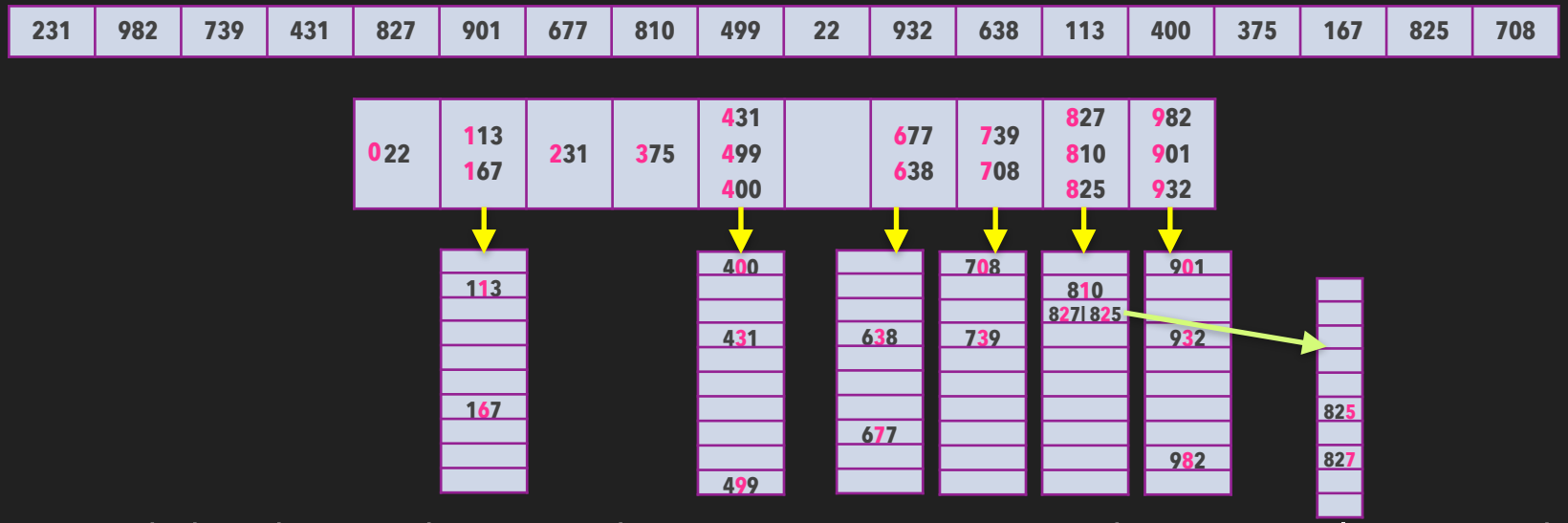
Radix Sort: MSD vs. LSD

Sorting

- Last time:
  - Counting sort: When the key-space is small, calculate the offsets for items with each key, and move them to the correct position
  - Bucket sort: Counting sort, applied to bucketed keys, followed by sorting each bucket
  - MSD Radix Sort: Bucket sort, with large buckets sorted recursively
- Today:
  - MSD Radix Sort (continued)
  - Least-Significant Digit (LSD) Radix Sort
  - MSD vs LSD Radix sort

Most-Significant-Digit Radix Sort

- Consider D-digit numbers
  - First collect by MSD



  - Each bucket with more than one item is sorted recursively using the remaining digits

Most-Significant-Digit Radix Sort

- What MSD Radix sort implements is *lexicographic sorting* (dictionary order)
  - E.g., `cab < car < dad < dog`
  - Strings needn't be the same length: `dog < dogma` (imagine an end-of-string character lesser than all other characters)
- For integers, to ensure lexicographic ordering is the same as numerical ordering, need 0-padding so that all numbers have the same number of digits
  - E.g., 22 < 113 numerically but not lexicographically; after 0-padding, lexicographically also, 022 < 113
  - "Digit" can be in any convenient base, e.g., base 256 allows each byte to be a single digit

Most-Significant-Digit Radix Sort

- There are B^D D-digit numbers (or D-long strings) in base B
  - This space is exponentially large in D, making counting sort impractical even for modest D
  - Also, comparison sort is expensive for large D, since a single comparison of two keys requires inspecting up to D digits, one by one
- MSD Radix sort costs:
  - Worst-case time and space: O(D(n+B))
    - Exercise: Prove by induction on D. Base case, D=1, is counting sort.

LSD Radix Sort

- What if we first sort the numbers by their LSD?

4012
1091
30
785
7703
4388
9452
92
1179
6905
3498

1000	100	10	1
4	0	1	2
1	0	9	1
0	0	3	0
0	7	8	5
7	7	0	3
4	3	8	8
9	4	5	2
0	0	9	2
1	1	7	9
6	9	0	5
3	4	9	8

1000	100	10	1
0	0	3	0
1	0	9	1
4	0	1	2
9	4	5	2
0	0	9	2
7	7	0	3
0	7	8	5
6	9	0	5
4	3	8	8
3	4	9	8
1	1	7	9

1000	100	10	1
7	7	0	3
6	9	0	5
4	0	1	2
0	0	3	0
9	4	5	2
1	1	7	9
0	7	8	5
4	3	8	8
1	0	9	1
0	0	9	2
3	4	9	8

1000	100	10	1
4	0	1	2
0	0	3	0
1	0	9	1
0	0	9	2
1	1	7	9
4	3	8	8
9	4	5	2
3	4	9	8
7	7	0	3
0	7	8	5
6	9	0	5

1000	100	10	1
0	0	3	0
0	0	9	2
0	7	8	5
1	0	9	1
1	1	7	9
3	4	9	8
4	0	1	2
4	3	8	8
6	9	0	5
7	7	0	3
9	4	5	2

30
92
785
1091
1179
3498
4012
4388
6905
7703
9452

LSD Radix Sort

- LSD Radix sort: Sorting all the numbers by their digits using counting sort, one digit at a time, starting from LSD
- Why does LSD work? Proof by induction:
  - Base case is the 1 digit case.
  - Enough to argue that any two numbers are correctly sorted relative to each other
    - If they differ in MSD, the last pass sorts them correctly
    - If they have the same MSD, then they were correctly sorted before the last pass (by induction) *and* *the last pass does not change their relative order* (we crucially need each pass to be a stable sort)

LSD Radix Sort

- Simply call counting sort in a loop, one digit at a time as the key
  - Here each item is a D-sized vector of B-ary digits

```
void LSD_Radix(std::vector<std::vector<int>>& A, int D, int B) {
  std::vector<std::vector<int>> aux(A.size());
  for(int d=0; d<D; ++d) { // counting sort using d-th digit (d=0 is LSD)
    std::vector<int> offset(B,0);
    for(auto& x : A) ++offset[x[d]]; // after this, offset has counts
    for(unsigned int i=0, sum = 0; i < B; i++) {
      unsigned int tmp = offset[i];
      offset[i] = sum; sum += tmp;    // offset[i] has sum of previous counts
    }
    for(auto& x : A) // aux[ offset[i] ] is where next item with key i goes
      aux [ offset[x[d]]++ ] = std::move(x); // move items from A to aux
    A.swap(aux); // finally move aux vector to A, keeping aux's size same
  }
}
```

MSD vs. LSD Radix Sort

- MSD version can eliminate short strings (or strings that fall into small buckets) early on. LSD version goes over every string in all D iterations.
  - MSD radix sort better suited for sorting variable length strings, or long strings that are lexicographically "far apart"
- MSD version requires more book-keeping, as sorting is applied to each bucket separately. LSD version is simpler and more cache-friendly.
  - LSD radix sort better suited for sorting fixed-length "strings" (e.g., 32-bit integers with exactly D = 4 digits, in base B = 2^8 = 256)
- Both are highly parallelizable, but the LSD version allows all processors working equally and in-sync
  - LSD radix sort (with optimisations) used for sorting integers in GPUs

Summary

- Counting sort
  - Useful when the key domain size k is small. O(n+k) time and auxiliary space
- Bucket sort
  - If data is "well-spread" over a few buckets into which keys can be partitioned
  - On well-spread data, using k buckets, O(n+k) time and auxiliary space. But worst-case time is O(n log n + k) (assuming O(1)-time comparisons).
- MSD Radix Sort
  - Using bucket sort recursively, for lexicographically sorting strings. Worst-case time and auxiliary space: O(D(n+B)) for D-long strings over an alphabet of size B. (Faster, if common prefix of every pair is short.)
- LSD Radix Sort
  - Using counting sort iteratively. O(D(n+B)) time and auxiliary space.
  - Parallel optimised version used for sorting numbers in modern GPUs

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 23

Radix Trees

A Data Structure for Storing Strings

- How can we store a set of strings and search in it efficiently?
  - Option we already have: Use std::set<std::string>
    - Comparing a pair of strings takes time proportional to the length of the (shorter) string
    - Searching, inserting, deleting in a set of n strings of length D takes O(D log n) time
  - Option : Use a radix tree
    - Searching, inserting, deleting in a set of n strings of length D takes O(D) time!

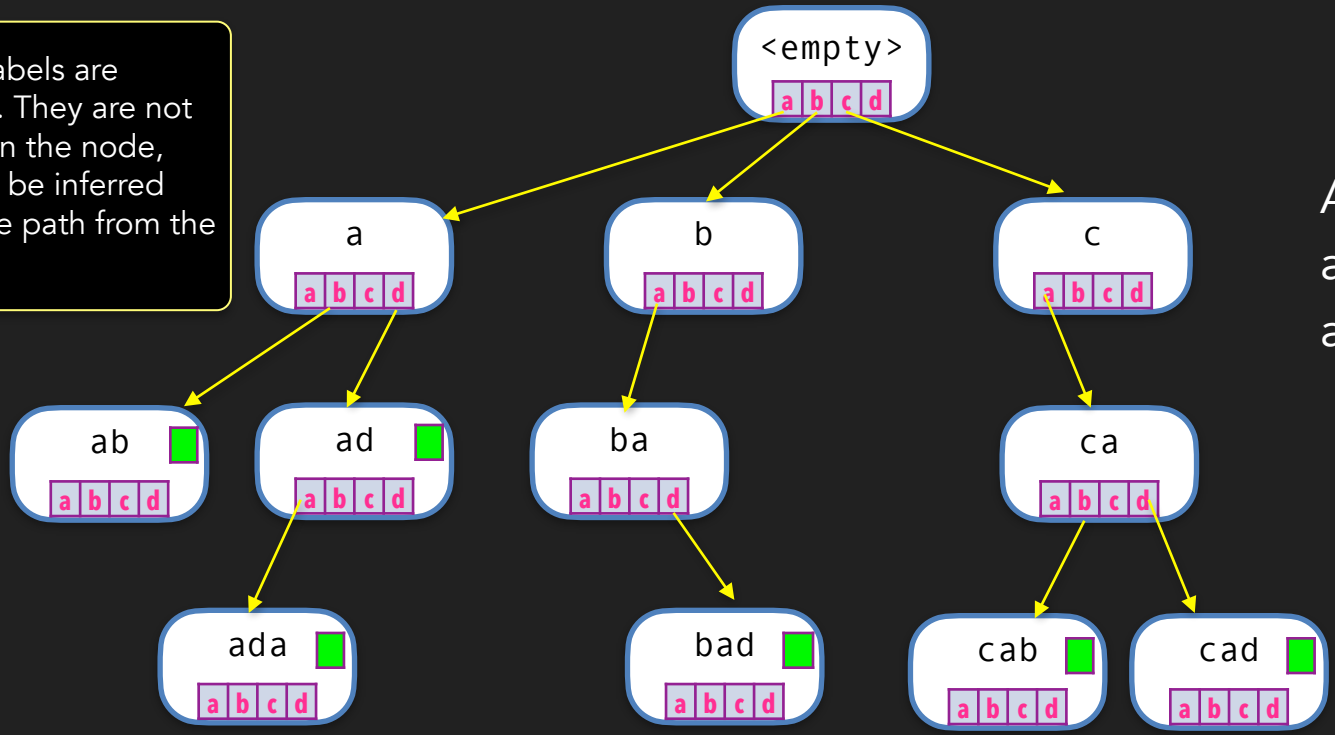
Trie

- A Radix Tree is an optimisation of a simpler data structure called a Trie
- For storing strings over an alphabet of size B:
  - A full B-ary tree: Each internal node has B children, each child labelled by a distinct character from the alphabet
  - The labels on the path from the root to a node is a string that is associated with that node
  - For each prefix of a string that is stored, there is a unique internal node. Leaves correspond to minimal strings which are not such prefixes.
  - Each node has a boolean flag `full_entry` indicating whether the corresponding string is stored, or if it is merely a prefix (alternately/ equivalently, include a special end of string character in the alphabet)

Trie

- Strings: { ab, ad ada, bad, cab, cad }. Alphabet: (a, b, c, d)

Node labels are implicit. They are not stored in the node, but can be inferred from the path from the root.

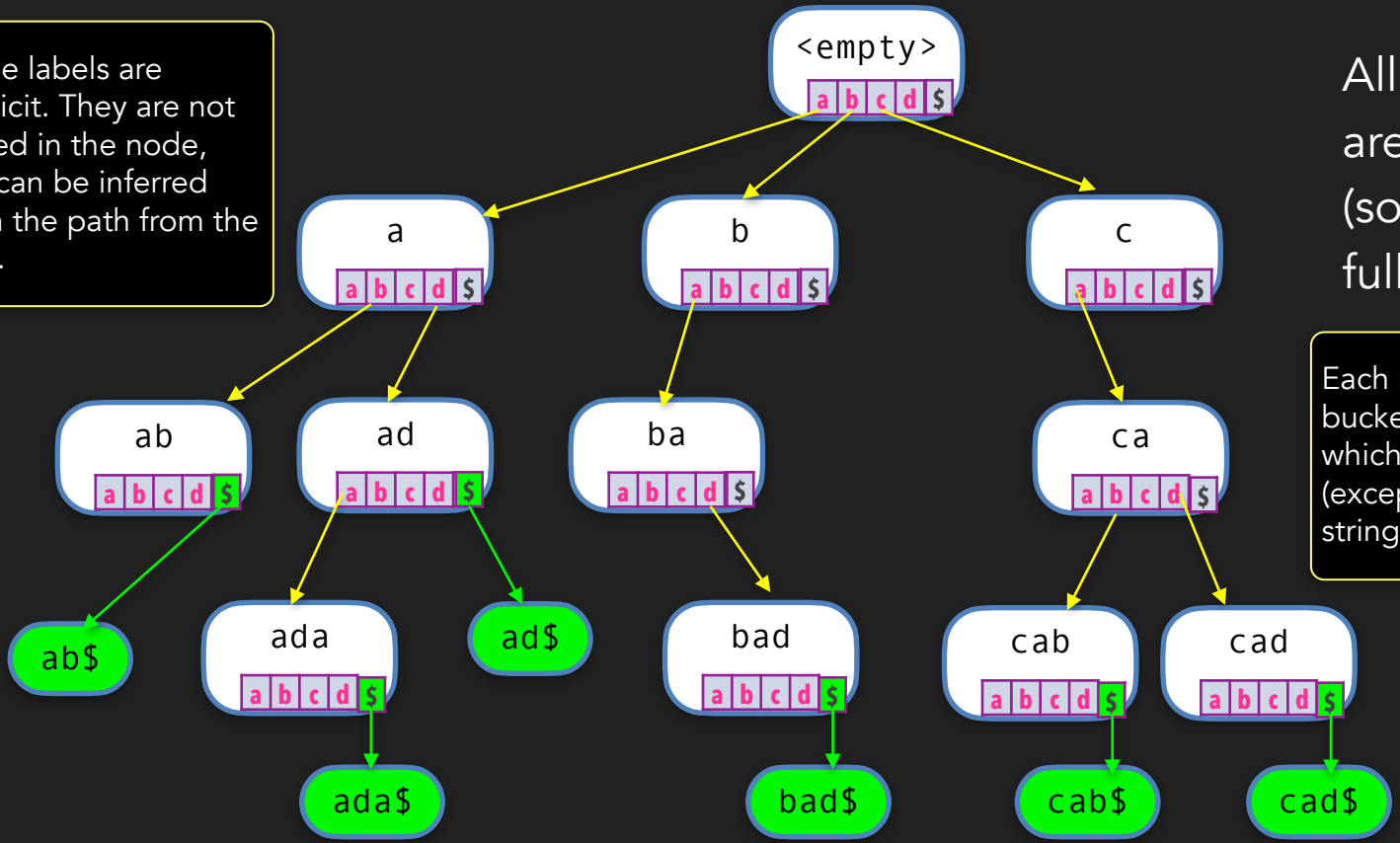


All internal nodes are prefixes. Some are also full entries.

Trie

- Alternate view: If `full_entry` set, has a leaf child that is a full entry

Node labels are implicit. They are not stored in the node, but can be inferred from the path from the root.



All internal nodes are prefixes. Only (some) leaves are full entries.

Each node corresponds to a bucket in MSD Radix Sort which is recursively sorted (except nodes with only one string left)

Trie

- Searching in a trie: Simply follow the unique path determined by the characters of the string ("MSD" first)
  - If the path exists, and further when the string ends, the full_entry flag is set (i.e., child for $ also exists), then the string is present
- Time: O(D) for searching D-long strings
  - Independent of how many strings are stored in the trie!
  - cf. Using std::set<std::string> requires Θ(D log n) time
- Can also search for all strings with a given prefix query, or the strings that share the Longest Common Prefix with a given query string
  - Important in many applications: Internet Packet routing tables, autocomplete from a dictionary (e.g., in IDEs, browser URLs, tab completion in shells, ...)

Trie

- Trie also supports easy insertion and deletion
  - Insertion: Search to find the longest prefix, and insert a chain of nodes from there
  - Deletion: Find the longest prefix which has at least two children, and delete all the nodes in the chain from there till the entry being deleted
  - Handle the full_entry flag correctly when inserting/deleting the leaf in these chains
    - Inserting the leaf node: just set the full_entry flag in the parent
    - Deleting the leaf node: just reset the full_entry flag in the parent

Attendance Break!

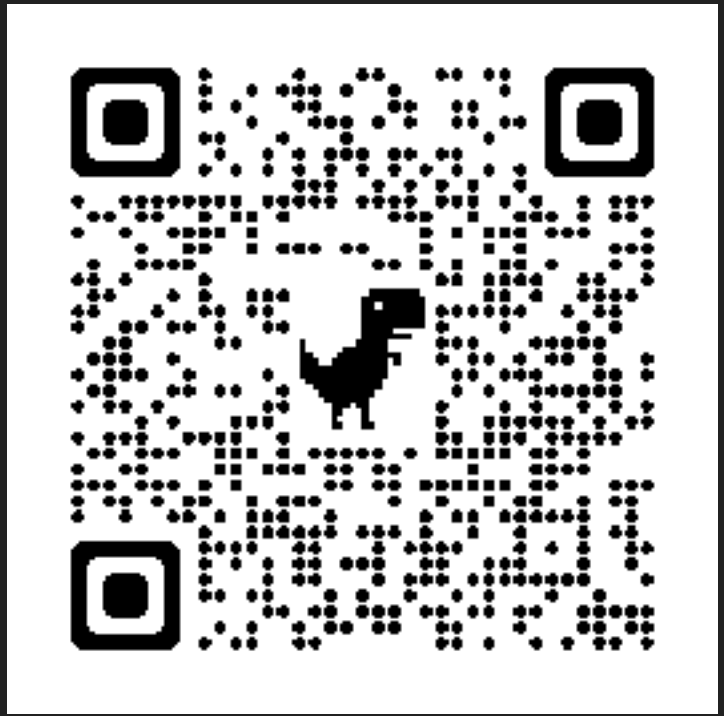
Login using CSE LDAP id at
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

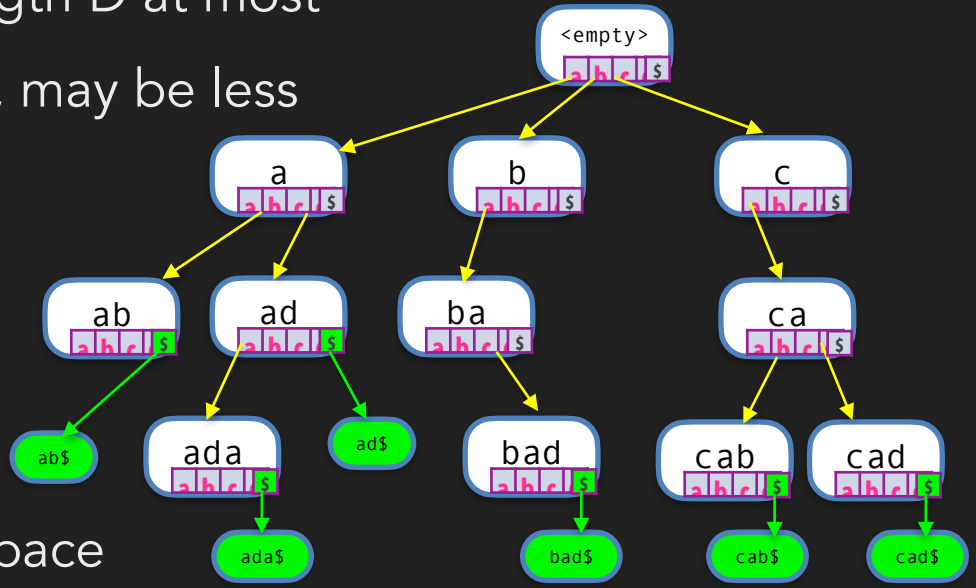
The selected students should come to the front

If you are not in the classroom, a report will be sent to DDAC!



Space in a Trie

- Each internal node takes O(B) space for the B child pointers
- O(nD) nodes for n strings of length D at most
  - By sharing common prefixes, may be less
- Total space: O(nDB)
- Compare to O(nD) for std::set<std::string>
- Can we do better?
  - Yes, two avenues of saving space
    - Don't use Θ(B) space for all nodes
    - Reduce the total number of nodes

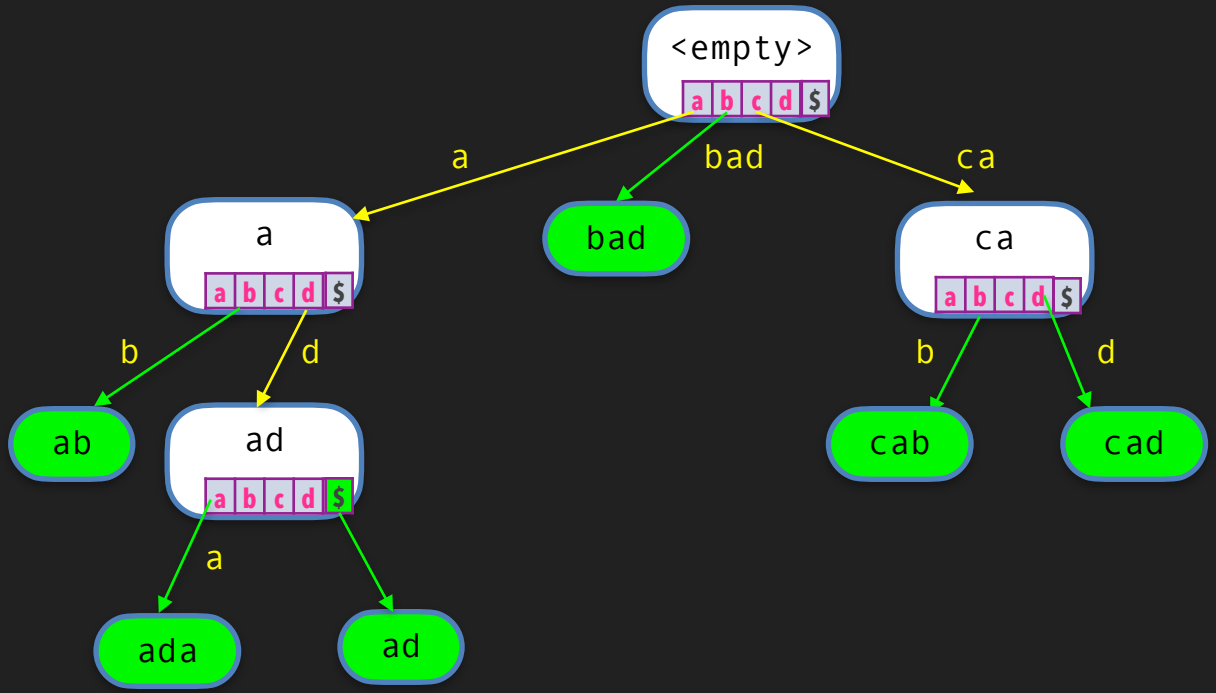


Reducing Space per Node

- Instead of O(B) space, we can use just O(#children) space
  - Use a map (e.g., BST as in std::map or Hash Tables as in std::unordered_map) to store pointer to each child, keyed using the alphabet of size B.
    - std::map (as opposed to std::unordered_map) changes lookup time from O(1) to O(log B), but retains ability to search for smallest string with a prefix, etc.
    - Can adaptively switch the representation, as the number of children changes: when the number of children is small, just store a sorted vector
- How many edges?
  - O(nD) nodes with O(B) children. O(nDB). No gain?
  - Count by the other end of the edge: O(nD) edges!

Reducing the Number of Nodes: Radix Tree

- Path compression: Each edge has a string of symbols, and there is no internal node with a single child (except possibly the root, when n=1)



Reducing the Number of Nodes: Radix Tree

- Path compression: Each edge has a string of symbols, and there is no internal node with a single child (except possibly the root, when n=1)
- How many nodes?
  - Recall: if every internal node has exactly 2 children, n leaves ⇒ n-1 internal nodes (proof by induction)
    - Trimming two leaves with the same parent reduces the number of leaves by 1 and number of internal nodes by 1
  - If every internal node has ≥ 2 children, n leaves ⇒ ≤ n-1 internal nodes
    - Trimming k ≥ 2 leaves reduces the number of leaves by k-1 and number of internal nodes by 1

Reducing the Number of Nodes: Radix Tree

- Path compression: Each edge has a string of symbols, and there is no internal node with a single child (except possibly the root, when n=1)
- How many nodes?
  - n leaves $\Rightarrow \leq$ n-1 internal nodes (except if n=1)
- Total number of nodes is now independent of the string length
- Edge-labels (stored in a map at the parent-node) take a total of O(nD) space
- O(n) space for tree, O(nD) space for edge-labels $\Rightarrow$ O(nD) total space
- O(D) or O(D log B) search time (depending on how the edge map is implemented)

Radix Tree

- Searching in a radix tree requires checking if edge label matches the remainder of the query string
- Inserting a string into a radix tree could require splitting an edge into two (where the mismatch with the inserted string occurs), and inserting a node in between (which then gets a second child corresponding to the inserted string)
- Deleting a string may cause a (non-root) node to have only one child left. Then the node is eliminated and its incoming edge and outgoing edge are merged.

Substring-Matching Using a Suffix Tree

- Goal: To search for query substrings in a long text
- A Radix Tree allows searching for prefixes of a set of strings
- Any substring of T is a prefix of some suffix of T!
- If we store all the suffixes of the text in a Radix Tree, we can query any substring!
  - Suffix Tree of T: A Radix Tree of all the suffixes of T
- There is an O(D) time algorithm ("Ukkonen's algorithm") for constructing a suffix tree for a D-long text
  - Subsequently, searching for a substring of length d takes O(d) time

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 24

Hash Tables

Motivation

- Suppose we are storing n distinct keys from a domain of size N
- Store them in an array A of size N
  - Store item with key i at A[i] (initially A[i] is marked "empty")
  - O(1) time for search/insert/delete!
- This is called Direct Addressing
- But $\Theta(N)$ space: realistic only when N is "small." Not feasible when the keys are say, integers ($N=2^{64}$ typically), or byte-strings of length 5 ($N=2^{40}$)
- Question: Can we match the speed of direct addressing using a reasonable amount of space?

Motivation

- Now, suppose we are storing n keys from a very large domain K
- Without direct addressing, we just need an array A of size m ≥ n to hold them
  - But searching would be slow, as a key could be sitting anywhere in the array
  - What if we could *calculate* its index as H(key) directly, using a "hash function" H: K → {0,...,m-1}?
  - When searching for key, we just compute H(key) and check if A[H(key)] has key stored there.
- Too good to be true! If |K| >> m, any function H: K → {0,..,m-1} will have to map many keys to the same index (collisions)
  - Hope: If we keep m slightly larger than n, then among the n items we get, there will only be a few collisions in each index. And, we can handle a few collisions.

Hash Tables

- Ingredients to make hash tables work
  - Good hash functions: how to ensure that there will only be a few collisions in each index (with high probability)
  - Handling increasing load: Grow the hash table as more keys inserted
  - Handling collisions: Two different paradigms
- Coming up today:
  - What are good hash functions

Hash Functions

- Central idea: If the function H is "random", and the load factor n/m is bounded by a constant (say < 1), then with high probability, there will be at most a few collisions in almost all indices
- What does with high probability mean?
  - There is a "universe of functions" $\mathcal{H}$ such that for any set of n keys chosen to be stored, *most* $H \in \mathcal{H}$ (say, more than 1-ε fraction) lead to low collisions
  - So if H is picked randomly from this universe, the probability that we pick a bad H (i.e., not with low collisions) is at most ε
    - However, for each H, there exist many key-sets with high collisions (in fact, if $|K| > mn$, there will be a key-set that is entirely mapped to a single index): No single H is universally good for all key-sets

Hash Functions

- What is a good universe of hash functions?
  - One that enforces "uniform hashing": each of the n keys is mapped to a uniformly random index, independent of the others
    - Then the expected number of items in any index is n/m, and further the probability that it is very far from expectation is very low
  - E.g., Universe of all functions from the domain of keys K to {0,...,m-1}
    - Picking a random H from this universe is exactly the same as uniform hashing (without limiting to n keys)
    - But no succinct description of a random function in this universe
  - We can "mimic" the universe of all functions using a smaller "pseudorandom" universe with efficiently representable functions

Hash Functions

- Pseudorandom: Has a similar effect as truly random.
  - Empirical (or Heuristic): Designed to work like random on a benchmark of inputs. No guarantees against adversarial inputs. Very fast.
  - Combinatorial: Provably as good as random against one-shot adversarial inputs. No guarantees against adaptive adversarial inputs.
  - Cryptographic: Strong guarantees. Only slightly slower, if relying on "lightweight-cryptography". Default in Python/Rust.
- Applications (e.g., those running on a web-server) which store untrusted inputs in hash tables are vulnerable to Denial of Service attacks, unless cryptographic hashes used

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 25

Hash Tables (ctd.)

Hash Tables

- Recall Hash Table template:
- Initialise an array A of size m, and a hash function H drawn from a family $\mathscr{H}$
- To search for the location of a key, compute an index H(key) $\in$ {0,...,m-1}
  - H(key) should appear to be uniformly random, independent of the other keys
  - Load factor n/m determines the expected number of collisions at each index. If it is too large, rehash: increase m, pick H from a corresponding $\mathscr{H}$, and reinsert all elements into a new table of size m
  - Need to handle collisions
- Today:
  - Rehashing, and handling collisions

Two-Stage Hash Functions

- Recall that to limit collisions, even using an ideal hash function, the load factor n/m should be upper bounded by a small constant (say 1)
- So, as n grows, m will have to grow too
- When m changes, the hash function will have to change too!
- This motivates a *two-stage hash function:* $H(key) = g_m(h(key))$, where $h: K \to R$ for a fixed range R (usually 64-bit integers) and $g_m: R \to \{0,..,m-1\}$
  - h is drawn from a good hash function family that doesn't depend on m
  - g_m is a (fixed, fast) *regular* function: i.e., sends a uniformly random input to a uniformly random output. Compresses $r \in R$ to an index.
- In some cases (e.g., C++ STL defaults) h is not good (i.e., not approximating uniform), but only collision resistant. Then g_m should ideally be drawn from a good hash function family (not so in STL!)

Two-Stage Hash Functions

- In std::unordered_set and std::unordered_map, while instantiating the template, one can/must provide a hash function h
  - Defaults are available for basic types, pointers, std::string etc.
  - But not for vectors or structs/classes
- The function g_m is fixed by the library for each m (e.g., modulo m)
  - In some implementations (with weak default h), m is chosen to be a prime number, to prevent g_m from creating collisions on regularly spaced inputs
- When m changes, the hash function $H = g_m \circ h$ needs to be redefined, and recomputed for every item in the hash table
  - h(key) will be *cached* along with the item, so that rehashing can avoid recomputing h, and only compute g_m

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Handling Collisions

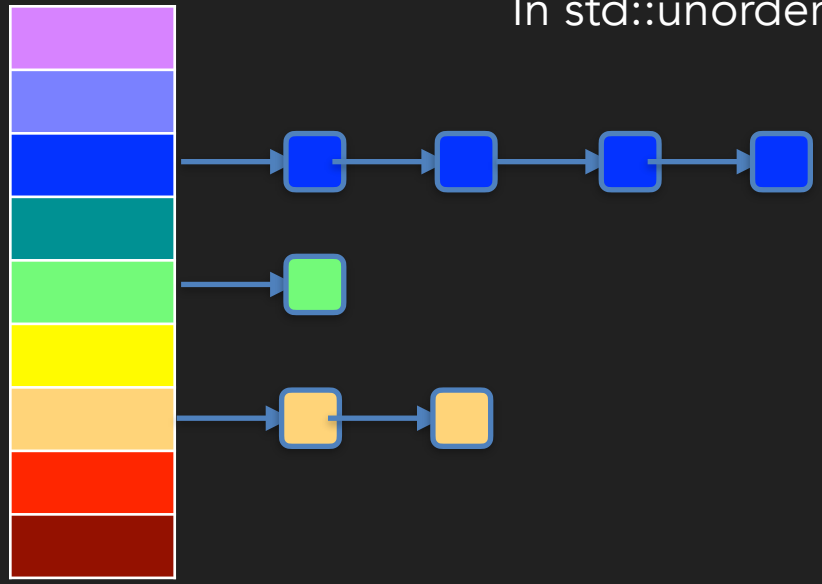
- What about the few collisions that do occur, even when we are lucky?
  - Simplest paradigm: At each index, allow multiple items to be held
  - Alternate paradigm: When collision, find another spot (next lecture)
- In std::unordered_map, items falling into the same index are held in a linked-list
  - Other containers would work too
    - E.g., in Java, switch to balanced BST when large number of items in an index (possibly because adversarial inputs broke uniform hashing assumption)
  - If the number of items in this container is O(1), looking through them will cost only O(1) time, and so total look up time is still O(1)
    - Assuming uniform hashing and bounded load factor, indeed O(1) expected list size, and further, with overwhelming probability, only at a few indices the number of items could be slightly larger: in $O(n^{\delta})$ indices O(log n/log log n) items

Handling Collisions

- A secondary goal in a Hash Table: O(n) time enumeration of all the items stored in the table (in arbitrary order)
  - So each bucket should be enumerable
  - Should be able to move to next bucket from the end of one bucket
- Idea: A single global list of all items, formed by lists from all the (non-empty) indices linked together
  - At each non-empty index, head and tail of its segment in the global list can be stored
  - When an index is populated for the first time, its list can be inserted at the head/tail of the global list

Chaining

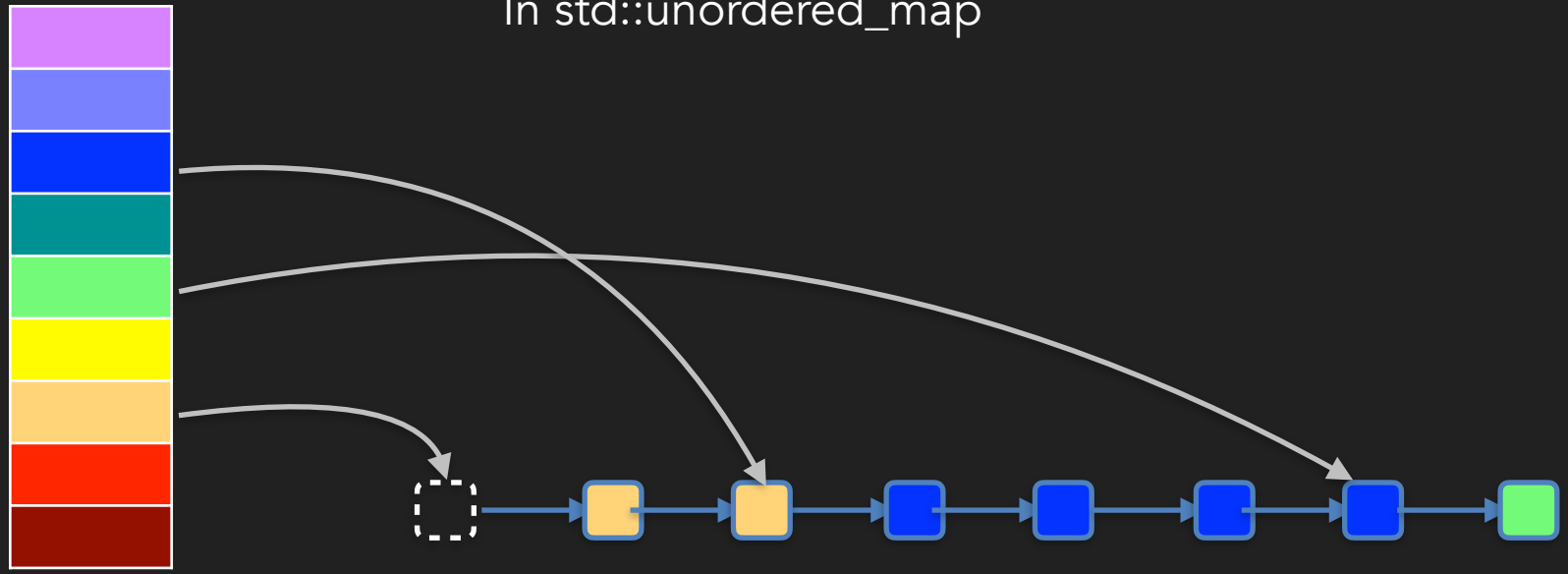
In std::unordered_map



Each index holds zero or more items, in a chain

Chaining

In std::unordered_map

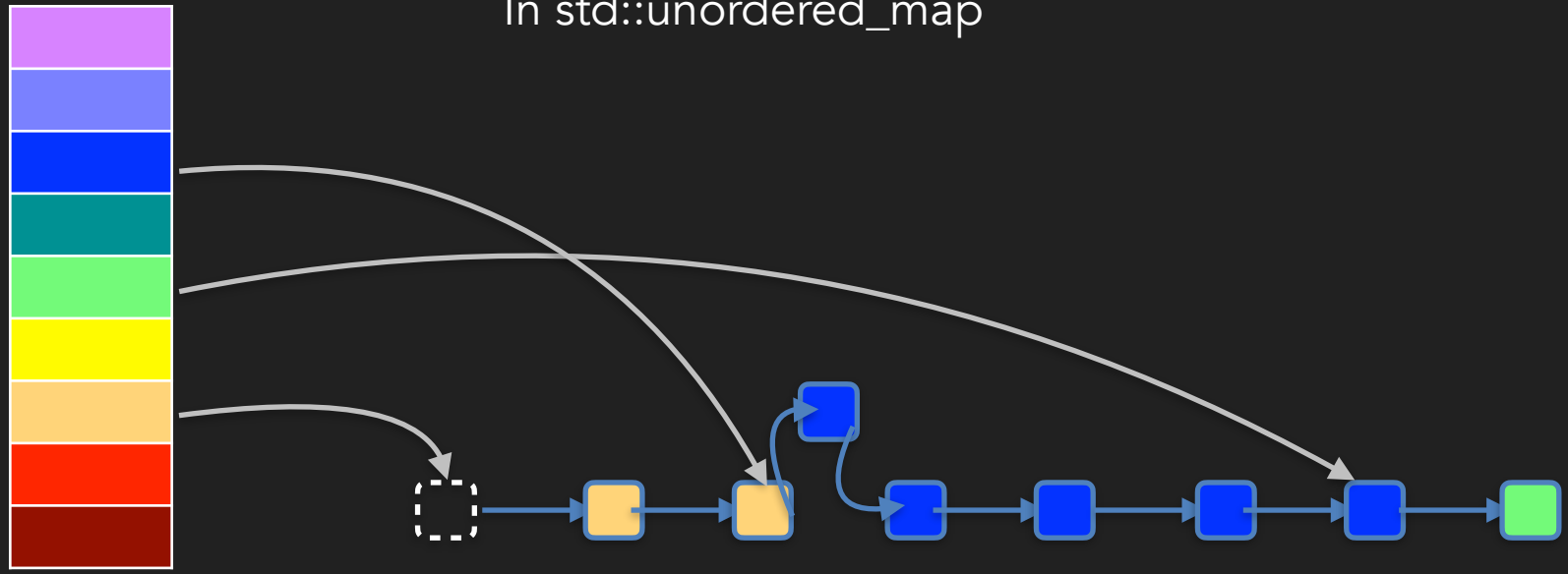


All items held in a single global list to enable O(n) time enumeration

While searching in a single sub-list, should be able to tell where it ends

Chaining

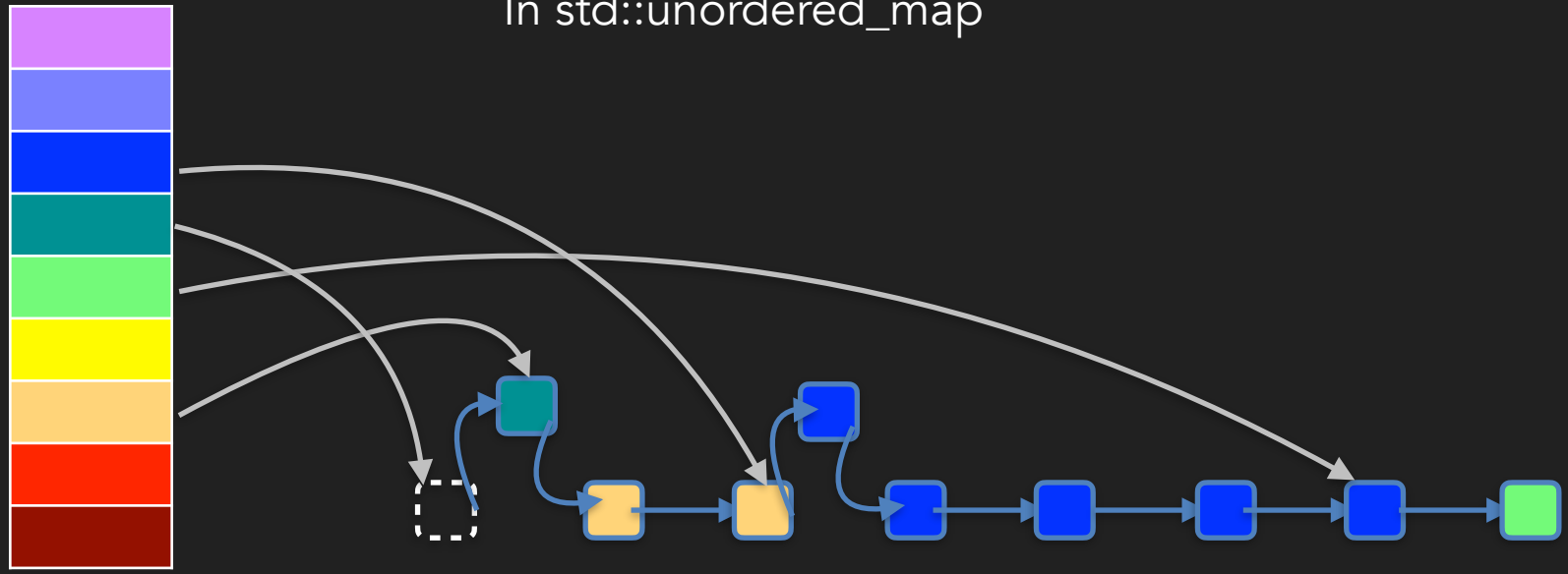
In std::unordered_map



New items at each index are added to the head of their sub-lists

Chaining

In std::unordered_map



New sub-lists are created at the head of the list

Optimisations

- In each item, store cached_tag = h(key), where $H = g_m \circ h$
- While searching in a list:
  - Compare cached_tags (integers) before comparing keys (which may be more complex)
  - Stop when g_m(cached_tag) changes -- it is the next bucket!
    - In each index store only a pointer to the start of its segment in the global list (or a nullptr): no need for storing the tail.
- When rehashing
  - Copy all items from the current global list into a new hash table, but while recomputing H, only need to compute g_m(cached_tag) now

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 26

Hash Tables

Open Addressing

Hash Tables

- Recall two paradigms for handling collisions:
  - At each index, allow multiple items to be held
    - The bucket holding the colliding items can grow arbitrarily, unless the hash functions are (cryptographically) strong
    - Items are scattered around in memory (linked together in a global linked list), negating cache benefits
  - When collision, find another spot in the hash table
    - Benefits from caching of the hash table
    - While inserting, may need to try multiple times before finding a spot. And while searching, will need to look at all those potential spots.

Closed Addressing

Open Addressing

Probing

- (Greedy) Probing template:

```
initialise an (implicit) sequence H₀(key), H₁(key), ...
for i = 0,...,N  // N can be set to m
   let t = Hᵢ(key)
   if inserting: // Assumes key not already present
                 if A[t] free: insert at A[t] and return
   if searching: if A[t] matches key:  return A[t]
                 else if A[t] never used: return not found
return no_space  // no space for key in the table
```

- Inserts at the first free location in the sequence (hence greedy)
- Later, when searching, guaranteed to look till the inserted location (will return early only if a slot that is never used is encountered, which insertion couldn't have skipped)
- If hash sequence is periodic mod m, N=m. If the hash sequence guaranteed to visit all m locations, N = m-#never used slots suffices: after N+1 probes guaranteed to see a never used slot.

Probing

- (Greedy) Probing template:

```
initialise an (implicit) sequence H₀(key), H₁(key), ...
for i = 0,...,N  // N can be set to m
   let t = Hᵢ(key)
   if inserting: // Assumes key not already present
                 if A[t] free: insert at A[t] and return
   if searching: if A[t] matches key:  return A[t]
                 else if A[t] never used: return not found
return no_space  // no space for key in the table
```



- To handle deletions, we mark a slot as never used, previously used (deleted), or currently used. Free slots are those which are not currently used.
  - Suppose key_1 is inserted at index t_1, followed by key_2 at index t_2. Also, suppose while inserting key_2, we encounter t_1 before reaching t_2.
  - Now suppose key_1 is deleted, and then we search for key_2. While looking for key_2 we should not stop at the free slot t_1.

Probing

- (Greedy) Probing template:

```
initialise an (implicit) sequence H₀(key), H₁(key), ...
for i = 0,...,N  // N can be set to m
   let t = Hᵢ(key)
   if inserting: // Assumes key not already present
                 if A[t] free: insert at A[t] and return
   if searching: if A[t] matches key:  return A[t]
                 else if A[t] never used: return not found
return no_space  // no space for key in the table
```

- To insert new elements, need the load factor to be < 1 (cf. closed addressing)
- Probing doesn't affect entries already in the hash table (later: probing with reordering)
- Efficiency determined by Probe-Sequence Length: number of iterations of the loop
- Above, inserting *assumed* key is not already present. If key present, should update instead of inserting a new item. Should do this without scanning the array twice.

Probing

- A more precise template. Initially, every slot in A has `(never_used,-)`.

```
Probe(key): // returns (FOUND,index) or (NOTFOUND,first free slot)
initialise an (implicit) sequence H₀(key), H₁(key), ...
first_free = -1
for i = 0,...,N  // N can be set to m
   let t = Hᵢ(key)
   if A[t] matches key:  return (FOUND,t)
   if A[t] deleted or never_used: if first_free == -1: first_free = t
   if A[t] never_used: break // end the probe
return (NOTFOUND, first_free)
```

```
Search(key):
(status, slot) = Probe(key)
if status == FOUND: return A[slot]
else return not_found
```

```
Delete(key):
(status, slot) = Probe(key)
if status == FOUND: A[slot] = (deleted,-)
```

Tombstones can slow down probe

```
Insert(item):
(status, slot) = Probe(item.key)
if slot != -1: if A[slot] never_used, ++N; A[slot] = (used,item); return SUCCESS
else return FAILED
```

Uniform Probing

- Uniform probing: An "ideal" that we shall approximate
  - For each (key,i), H_i(key) is a fresh random index
- Expected probe-sequence length:
  - Suppose load factor is α = n/m. Each probe has a probability α of failing (i.e., landing in an occupied slot) and probability (1-α) of succeeding.
  - Expected probe-sequence length $= \sum_{i=1}^{\infty} i \times \Pr[\text{took } i \text{ tries}]$

$$= \sum_{i=1}^{\infty} i \times \alpha^{i-1}(1-\alpha) = \frac{1}{1-\alpha}$$

$\Theta\left(\frac{1}{1-\alpha}\right)$ is the best possible for any greedy, non-reordering probing strategy

  - Note: When α approaches 1, expected number of probes spike

Attendance Break!

Login using CSE LDAP id at

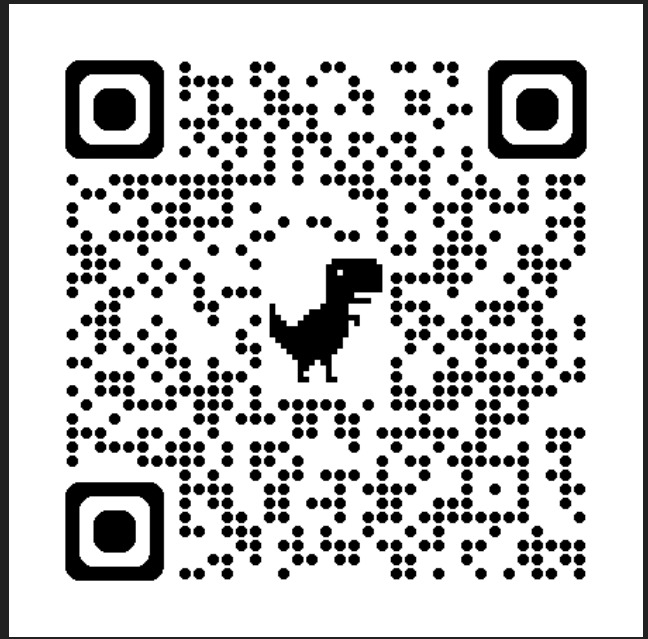
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Linear Probing

- Linear probing: $H_i(\text{key}) = H_0(\text{key}) + i \pmod m$ where H_0 is supposed to be a uniform hash
  - Simple: Can compute $H_i(\text{key})$ by simply incrementing (mod m)
  - Cache-friendly: Inspected locations will mostly be in the cache
  - But if $H_i(\text{key}_2)$ runs into $H_0(\text{key}_1)$, then it will have to traverse the segment already traversed by key_1
  - Expected probe-sequence length = $\Theta\left(\left(\frac{1}{1-\alpha}\right)^2\right)$
    - For α close to 1, much larger than in uniform probing

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 27

Hash Tables

Open Addressing (Ctd.)

Open Addressing

- Open addressing: Probe the hash table at a sequence of indices H_0(key), H_1(key), ...
  - Until reaching a location that has never been used
  - For inserting, first free location along the way is used (greedy)
  - In practice, probe sequence designed so that it reaches all indices in m steps
- Uniform Probing: For each (key,i), H_i(key) is independent random (unrealistic)
  - Expected Probe-sequence length: $\Theta\left(\frac{1}{1-\alpha}\right)$
  - Optimal for greedy, non-reordering probing strategy
- Linear Probing: $H_i(\text{key}) = H_0(\text{key}) + i \pmod m$
  - Expected Probe-sequence length: $\Theta\left(\frac{1}{(1-\alpha)^2}\right)$

Quadratic Probing

- Quadratic probing: $H_i(key) = H_0(key) + i^2 \pmod m$
  - Still simple: increment H_i by 2i+1 (increment the increment by 2)
  - Somewhat cache-friendly: Initially, for small i, inspected locations will mostly be in the cache
  - Expected probe-sequence length close to uniform probing for small α
- The above basic quadratic probe sequence can visit at most half of the indices (which happens when m is prime). Alternatives visit all indices:
  - When $m=2^k$, $H_i(key) = H_0(key) + i + 2\,i^2 \pmod m$ covers all indices. So does $H_i(key) = H_0(key) + i(i+1)/2 \pmod m$.
  - When m is a prime s.t. m-1 is not a multiple of 4, $H_0(key) \pm i^2 \pmod m$

Double-Hash Probing

- Double-Hash probing: $H_i(key) = H_0(key) + i \times H_\Delta(key) \pmod m$
  - Requires computing two hashes instead of one
  - Not cache friendly: Steps of size $H_\Delta(key)$ while probing
  - Probe-sequence length behaviour very close to that in uniform probing
- If $H_\Delta(key)$ is co-prime with m, this sequence reaches all m indices
  - E.g., $m=2^k$ and $H_\Delta(key) = 2H_\delta(key)+1$
  - Because 0, a, 2a, ..., (m-1)a (mod m) is a permutation of 0,1,...,m-1 whenever gcd(m,a)=1
  - Why? If $ai=aj \pmod m$, a(i-j) is a multiple of m. Since a has no factors of m, i-j has to be a multiple of m. Since $|i-j| < m$, we need i-j=0

Hybrid Probing

- We can consider a hybrid version that combines linear probing with another probing strategy h_0, h_1, ...
- Probe sequence: $T \cdot h_0(key)$, ... , $T \cdot h_0(key) + T\text{-}1$, $T \cdot h_1(key)$, ... , $T \cdot h_1(key) + T\text{-}1$, ...
- In modern hardware, can carry out a match on a small set of memory locations (T here) in one step
  - Match a one-byte *fingerprint* of the key (just another hash, typically implemented as a few reserved bits of h_0) against the fingerprints of the T locations. Look for a key match iff fingerprint matches.
  - Similarly, in one step can find a free slot out of the T slots
- Also, cache locality benefits of linear probing, along with probe-sequence length of the other probing strategy

Attendance Break!

Login using CSE LDAP id at

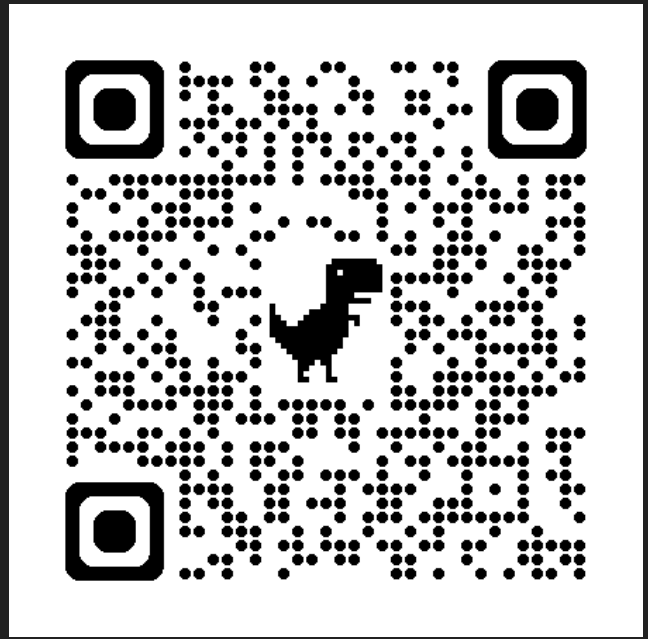
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Beating Uniform Probe

- For greedy probing, the best probe-sequence lengths were by uniform probe (or double-hash probe)
  - Expectation $\frac{1}{1-\alpha}$, and about 5% probes are over 3x the expectation
- Can we do better?
- Yes, by allowing reordering while probing!
- We'll see two schemes:
  - Robin Hood hashing: Take from the rich and give to the poor. Doesn't change the average, but virtually eliminates unlucky probes.
  - Cuckoo Hashing: Push out and occupy. Insert takes longer, but search is O(1), irrespective of the load.

Recent discovery: Also, possible without reordering, by not being greedy

Robin Hood Hashing

- Modified linear probing. Every item stores how far they are from their "home": `PSL(i) = i - h0(A[i].key) (mod m)`. The "rich" are close to their home.
- Inserting: Start linearly probing from the home index i_0, with `myPSL(i0)=0`.
  - As we are probing, the item we are holding gets poorer (`myPSL(i+1)=myPSL(i)+1`)
  - At an index i, if the item sitting there is richer than the item we are holding, (`PSL(i) < myPSL(i)`) then we swap the two. The richer item that is evicted continues down the linear probe path (getting poorer as it goes).
  - Continue till we find a free slot for the item we are holding.
- Searching: Stop when we encounter items richer than us.
- Deleting: Left shift each item to the right of the deleted item by one, till we encounter an empty slot or an item at its home. No tombstones!

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 28

Hash Tables

Open Addressing (Ctd.)

Open Addressing

- Open addressing:
  - Probe the hash table at a sequence of indices H_0(key), H_1(key), ...
- Probing without reordering: Uniform, linear, quadratic, double-hash
  - Hybrid: quadratic or double-hash probing combined with "parallel linear" probing
- Probing with reordering: Designed to "beat" Uniform Probing
  - Robin Hood hashing: Modified linear probing. Expected time for (successful or unsuccessful) search is $O(\frac{1}{1-\alpha})$, for insertion $O(\frac{1}{(1-\alpha)^2})$. Successful searches stay within an additive constant of expectation, and unsuccessful ones within O(log n) w.h.p.
  - Cuckoo hashing: O(1) worst-case search time. Insertion O(1) for small enough α.

Cuckoo Hashing

- We use d hash functions $H_1,\ldots,H_d$ where d is small (think of d=4)
- If key is stored, it will be at index $\in \{ H_1(key),\ldots,H_d(key) \}$: O(1) lookup time!
- Insertion algorithm, cuckoo_insert(item, $H_1(key),\ldots,H_d(key)$):
  - Look for an empty slot in the given set. If found, insert item there.
  - If no empty slot:
    - insert into one of the given slots (at random), "evicting" the item already there
    - recursively, cuckoo_insert the evicted item into one of its other d-1 slots
- Key question: (how fast) will cuckoo_insert terminate?

Cuckoo Hashing

- Only with α^d probability do we need to start on the eviction path
  - E.g., at α=0.8, d=4, about 0.4 probability
- Once we do start on the eviction path:
  - For a given d, there is a critical load factor α_{max}
    - For d=4, $\alpha_{max} \approx 0.97$. For d=2, $\alpha_{max} \approx 0.5$.
  - If $\alpha < \alpha_{max}$ probability of needing an additional step $\approx \alpha_{effective} := \frac{\alpha}{\alpha_{max}}$
    - Expected number of steps $\approx O(\frac{1}{1 - \alpha_{effective}})$
  - If $\alpha \geq \alpha_{max}$, almost surely goes into an infinite loop!

Cuckoo Hashing

- α_{max} is a "phase-boundary"
  - For $\alpha < \alpha_{max}$ we terminate fairly quickly
    - If $\alpha < \alpha_{max} - \varepsilon$ (where $\varepsilon > 0$ a constant independent of n) then expected number of steps during insertion = $O(1/\varepsilon) = O(1)$, and highly unlikely we go, say, 5x above the expectation
  - For $\alpha \geq \alpha_{max}$, if we start on the eviction path, we almost surely go into an infinite loop!
- If insertion takes too many steps, then we we're very likely to be in the infinite loop phase (assuming a cryptographically strong hash)
  - Fix: Reduce α. i.e., grow the hash table (double it) and rehash
- Alternately, before α gets too close to α_{max}, grow and rehash (and use a cryptographically strong hash)

Attendance Break!

Login using CSE LDAP id at

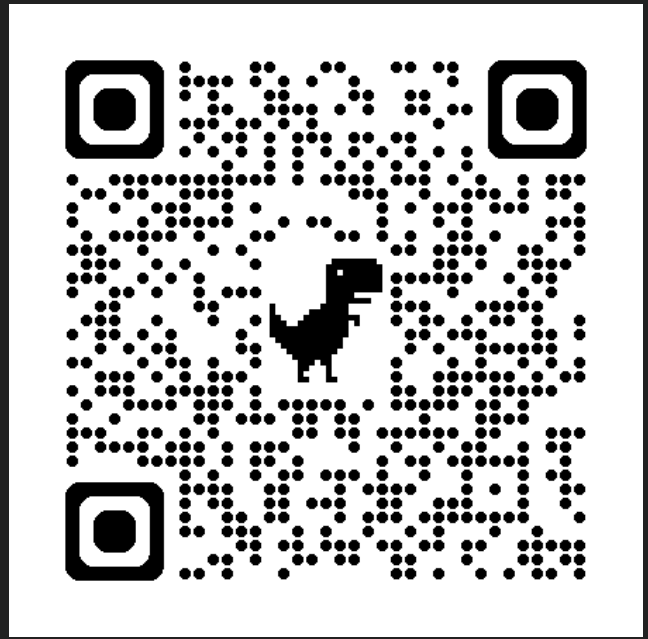
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Cuckoo Hashing with Parallel Checks

- Recall "parallel linear" probing:
  - Hash function yields an index to a "bucket" of B items
  - Fingerprints of all B items are checked in parallel (supported by hardware) against the current key's fingerprint. Full key checked only when fingerprint matches
- Extends to cuckoo hashing too: d hashes yield indices for d B-sized buckets. All Bd slots can be checked in one cycle.
- By increasing B, we can decrease d, without affecting α_{max}
  - E.g., d=2, B=4 has almost the same effect as d=4, B=1
  - Lower d is better (fewer hash functions to evaluate, fewer random locations to fetch), and a slightly large B is for free (still only one cycle)

Handling Adversarial Inputs

- Efficiency guarantees break down if the hash function doesn't behave like a uniformly random function (possibly under adversarial inputs)
- Can be prevented by using (light-weight) cryptographic hash functions
  - Against non-adaptive adversaries, enough to use a combinatorial hash function
- Alternately, can use a fallback mechanism on encountering a long eviction path/ probe path when load factor is small
  - Migration: Migrate the hash table to an alternate data structure (e.g., balanced BST, or a closed-addressing hash table with BST for each bucket)
  - Spillover stash: Add the input in hand to a stash implemented using an alternate data structure. When searching, first search in hash table, and then in the stash.

Hash Table Summary

- Main feature: practically O(1) lookup
  - In cuckoo hashing, guaranteed worst-case O(1)
  - In other open addressing schemes, expected O(1) time, assuming uniform hashing and $\alpha < 1 - \varepsilon$
    - In Robin Hood hashing, stays very close to $O(1/\varepsilon)$ for successful searches w.h.p. (and O(log n) for unsuccessful searches)
  - In closed addressing, assuming uniform hashing and $\alpha = O(1)$, all buckets remain practically O(1) size (technically $\Theta(\log n / \log \log n)$ w.h.p.)
- Also O(1) amortised insertion time
  - Insertion takes at least as much as an unsuccessful search, plus O(n) time when rehashing

Hash Table Practical Considerations

- Cache benefits
  - Looking up main memory several times adds a significant delay if the items are not cached. Using open addressing with a "hybrid" scheme avoids this delay.
    - Hybrid schemes for double-hash/quadratic probing and for cuckoo hash
    - Robin Hood hash is based on linear probing and avoids this delay.
    - In fact, closed addressing could also use chains of B-long vectors
- Parallelisation: Can check fingerprints of many slots in one step
- Adversarial inputs
  - Guarantees break down if the hash function used is not from a uniform hash family, and the inputs are crafted to exploit that
  - Can be avoided using (light-weight) cryptographically strong hash functions
  - Otherwise, in closed addressing, can fall back to BST for large buckets; in open addressing, migrate to BST/closed addressing table, or use a spillover stash